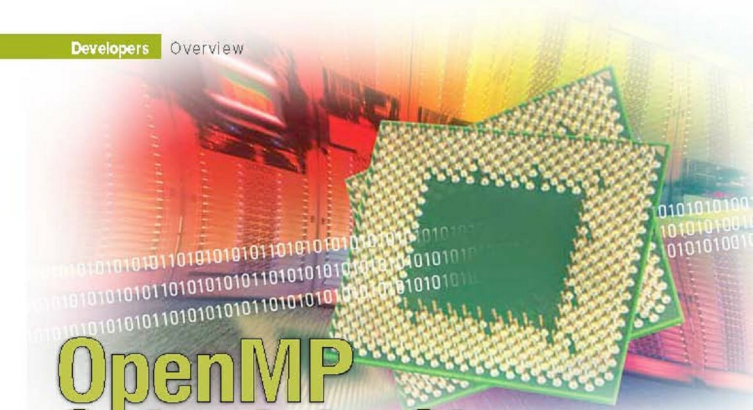




OpenMP

Schedule Clause

Parallel Matrix Multiplication

Even in this multi-core era, traditional programming methods don't take full advantage of multi-core systems. To do that, applications must use threads to execute code in parallel. OpenMP, an API for writing multi-threaded programs in a relatively easy way, can run on a variety of single- and multi-processor systems. It uses the Shared Memory Model, and mainly comprises compiler directives and environment variables used in C, C++ and FORTRAN. It is widely available across various architectures and operating systems. In this article, I will discuss how to use the OpenMP API to speed up an operation.

Let's cover a few basics about OpenMP before proceeding to our Matrix Multiplication operation. OpenMP is designed keeping in mind the perfect UMA (Uniform Memory Architecture) system, where all memory access latencies are the same. However, a NUMA (Non-Uniform Memory Architecture) system has variable latencies, depending upon the distance and the design topology used. I have used a NUMA system here for benchmarking. Since the detailed discussion of NUMA is beyond the scope of this article, please refer to Wikipedia and other resources.

Thread placement can be controlled using various CPU affinity features of the OS, as well as OpenMP. In Linux, NUMA systems threads are placed on CPUs closer to the allocated memory region, to avoid high memory access latencies. This is dynamically controlled by the Linux kernel, unless specified by the user. OpenMP version 4.x will have

better support for NUMA systems. Code can contain both serial and parallel statements and all the ugly syntax of threading is hidden behind the OpenMP *pragma* directives. OpenMP is best for loop-level parallelism, without many critical sections. Threads will be forked according to the *pragma* directives and joined after completion.

Using OpenMP, we can construct multiple parallel regions within the same code. Multiple threads can be forked on the occurrence of every parallel region construct. Thread creation involves certain overheads, which must be taken into consideration while writing scalable parallel code. OpenMP allows us to fork threads once, and then distribute nested parallel regions on these threads, reducing the overhead of thread creation on every parallel construct. Operations such as *flush()* help us to maintain data integrity between nested threads. OpenMP usually distributes one thread per core. More threads per core can